

AI 学习深化指南

配套《个人定制学习路线》· AI 方向细节展开

适合人群：有 Python 基础、目标是 AI 应用工程师
深化方向：API 实战 → Prompt 工程 → RAG → Agent → 微调 → 工程化
参考周期：与主路线阶段三同步推进，可并行启动

一	心态与认知：避开 5 大新手陷阱
二	阶段 1：LLM API 实战（第 1-2 周）
三	阶段 2：Prompt 工程深化（第 3-6 周）
四	阶段 3：RAG 系统从 0 到 1（第 7-12 周）
五	阶段 4：Agent 与工作流（第 13 周后）
六	模型本地化与微调（按需深化）
七	AI 工程化生产实践
八	踩坑经验集（12 个真实问题）
九	6 个进阶练手项目
十	精选资源清单（少而精）

一、心态与认知：避开 5 大新手陷阱

AI 学习失败的 80% 原因不是技术，是路径错误。先解决"怎么学"再解决"学什么"。

5 大陷阱清单

陷阱 1：原理先行	从 Transformer 论文、注意力机制开始，三个月后还在看公式，没写过一行 AI 代码。正解：先调通 API 做出能用的东西，原理在做项目时按需补。
陷阱 2：囤课不动手	收藏夹有 50 门 AI 课程，加起来 200 小时，看完前 10%。正解：一门课配一个项目，看一节做一节。
陷阱 3：框架先行	上来就学 LangChain，被抽象层绕晕，不知道底层在做什么。正解：先用裸 Python + requests 调 API，理解链路后再上框架。
陷阱 4：追新焦虑	GPT-5 出来换 GPT-5，Claude 5 出来换 Claude 5，Gemini 出来换 Gemini，永远在试模型，没产出。正解：固定一个模型（推荐 DeepSeek/Claude）做完一个项目再换。
陷阱 5：怕过时	担心 LangChain 明年没人用、ChromaDB 会被淘汰，于是什么都不学。正解：迁移成本极低，工程能力是通用的，模型/框架只是工具。

核心 3 原则

用中学	所有学习都围绕"做一个能用的东西"展开。学不动了就是项目还不够具体。
窄而深	一个方向打透再扩散。先把"调 API + Prompt"用熟，再碰 RAG。同时学三件事 = 一件都学不会。
公开输出	GitHub 提交、博客笔记、社区分享。外部反馈是最好的进度条。学了不输出 = 没学。

一句话总结：把"我要学 AI"换成"我要做 X"。X 越具体，学习路径越清晰。

目标：从 0 到 1 跑通 LLM API，理解所有核心概念，做出第一个能用的 AI 工具。

API 平台选型对比（2026 年）

平台	价格（百万token）	优势	场景
DeepSeek	¥1-8	便宜、中文好、国内直连、OpenAI 兼容	入门首选、日常调试
Claude (Anthropic)	\$3-15	长文本最强、代码最准、Tool Use 标准	复杂任务、Agent
OpenAI	\$0.15-15	生态最全、模型最多、SDK 最成熟	学习参考、英文场景
通义千问 (阿里)	¥0.5-20	中文场景、企业生态、阿里云集成	国内商业项目
智谱 GLM	¥1-50	中文长文本、清华系	国内备选
Ollama (本地)	免费	隐私安全、零成本、可离线	原型/隐私敏感

新手建议：DeepSeek 作为主力（便宜到可以随便实验），Claude 作为对照（理解一线模型水平）。不推荐一开始用 OpenAI——需要海外卡 + 代理，门槛把人卡在第一步。

API 6 大核心概念

messages	对话数组，每条带 role（system/user/assistant）。关键：模型本身无记忆，多轮对话靠你把历史塞回去。
role: system	系统提示词，定义 AI 的角色、能力、约束。决定 80% 的输出质量。
role: user / assistant	user = 你的输入；assistant = AI 的回复。多轮对话交替追加。
token	AI 处理的最小单位，约 1 中文字 ≈ 1.5 token，1 英文词 ≈ 1.3 token。所有计费、上下文长度限制都按 token 算。
temperature	0-2，控制随机性。0 = 确定（适合代码/JSON），0.7 = 平衡（聊天），1.5 = 创意。
max_tokens	限制单次回复最大 token 数。设太小会被截断，设太大浪费钱。常用 1000-4000。

最小可运行代码（Python）

```
from openai import OpenAI
client = OpenAI(api_key="sk-...", base_url="https://api.deepseek.com")

# streaming + multi-turn chat
history = [{"role": "system", "content": "You are a concise helper."}]
while True:
    q = input("You: ")
    history.append({"role": "user", "content": q})
    stream = client.chat.completions.create(
        model="deepseek-chat", messages=history, stream=True)
    reply = ""
    for chunk in stream:
        delta = chunk.choices[0].delta.content or ""
        print(delta, end="", flush=True); reply += delta
```

```
history.append({"role":"assistant","content":reply})
```

二、阶段 1（续）：7 天行动清单

第 1-2 周

Week 1：跑通 + 理解（每天 2 小时）

Day	任务	产出
1	注册 DeepSeek，充值 10 元；跑通 hello world 第一次调用	一段成功的对话日志
2	理解 messages 数组，做命令行多轮对话脚本	能记住上下文的 CLI 聊天
3	加入流式输出（streaming），实现打字机效果	体验接近 ChatGPT
4	玩 temperature/top_p/max_tokens 各种参数	记录不同参数的效果差异
5	做一个"AI 翻译"工具（输入中文 → 英文，并解释难点）	一个自己会用的小工具
6	做一个"代码注释生成器"（输入 .py 文件 → 输出带注释版本）	能跑通真实文件读写
7	总结笔记：API 调用全流程、踩过的坑、token 计费认知	一篇 500 字的复盘

Week 2：综合实战

Day	任务	产出
8-9	做一个"PDF 总结器"：读 PDF → 分段 → AI 总结 → 输出 Markdown	一个真实可用的工具
10-11	做一个"邮件起草助手"：输入要点 → AI 生成正式邮件，多版本对比	理解 prompt 的威力
12-13	把上述工具改造成 FastAPI 接口 + 简单 HTML 前端	为全栈集成做准备
14	部署到云服务器，做成可访问的 URL	第一个上线的 AI 应用

阶段 1 结束自检

能解释 messages / role / token / temperature 的作用

能不查文档写出一个流式对话脚本

理解为什么模型"没有记忆"（无状态）

能估算一次调用的 token 数与成本

至少做出 2 个自己真的会用的工具

目标：掌握工业级 Prompt 设计方法。Prompt 质量决定 AI 产品质量的 70%。

Prompt 黄金 5 段结构

1. 角色 Role	你是一个 10 年经验的 X 领域专家。给 AI 戴上身份，输出风格立即收敛。
2. 任务 Task	你的任务是 [具体动作 + 输入对象 + 输出对象]。越具体越好。
3. 约束 Constraint	禁止做什么、必须包含什么、长度限制、风格限制。
4. 示例 Example	给 1-3 个输入输出范例（few-shot），让模型抄"格式"。
5. 格式 Format	指定 JSON / Markdown / 表格 / 纯文本。结构化输出便于程序消费。

8 大核心技法

Role Prompting	设定专家身份。"你是资深刑事律师"vs"请帮我看合同" — 前者输出深度高一个数量级。
Few-shot	示例胜过描述。要让 AI 输出特定格式，给 2 个例子比写 500 字解释有效。
Chain of Thought (CoT)	加一句"请逐步思考"。复杂推理任务准确率提升 20-50%，本质是让模型多花 token。
JSON Mode / 结构化输出	强制输出 JSON 便于程序解析。Prompt 里加 schema，加"必须返回合法 JSON"。高级模型支持 response_format={"type":"json_object"}。
Prompt Chaining	拆任务。复杂任务 → 拆成 3-5 个简单 prompt 串起来。每步输出作为下一步输入。
Self-Reflection	让 AI 检查自己。第一次回答 → "现在请检查上述回答有无错误并改进"。质量普遍提升。
Meta Prompting	让 AI 写 Prompt。"请帮我设计一个用来 X 的 system prompt"。元能力，省时间。
Negative Prompt	明确禁止。"不要使用 markdown / 不要说抱歉 / 不要超过 100 字"。比期待模型猜你想法靠谱。

Function Calling / Tool Use 实战

让 AI 调用你的代码，是 Agent 的基础。流程：定义工具（JSON schema） → 模型决定调用哪个 → 你执行 → 把结果塞回去。

适用场景	查数据库、查实时数据（天气/股票）、调用第三方 API、文件操作、计算器。
关键认知	模型不会真的调用工具，它只是"建议你调用"。执行权在你手里。
设计原则	工具名 + 参数描述要写得像给 5 岁小孩看的说明书。模型理解你的工具全靠 description。
调试技巧	把 AI 的 tool_calls 输出打印出来看，90% 的问题都是参数 schema 没写清楚。
推荐学源	Anthropic 官网 tool_use 文档（最权威）+ OpenAI function calling cookbook。

防幻觉 5 招

给资料，不靠记忆	让 AI 基于你提供的文本回答，不要让它"凭印象"。这是 RAG 的核心动机。
允许说"不知道"	Prompt 里加："如果信息不足，必须回答'我不知道'，禁止编造。"
引用原文	要求 AI 回答时必须引用资料原句，方便人工核对。
降低 temperature	事实类查询用 temperature=0，减少创造性发挥。
双模型互检	答案让另一个模型审一遍。对高准确率场景有效（成本翻倍）。

Prompt 测试与迭代方法论

建测试集	准备 10-30 个典型输入 + 期望输出。改 prompt 后跑一遍，看哪些 case 退化了。
版本管理	Prompt 用 git 管，每次改动写 commit message 说明改了什么、为什么。
A/B 对照	改 prompt 时保留旧版本，新旧各跑 10 次取平均，避免抽样偏差。
用工具	LangSmith / LangFuse / PromptLayer 提供专业 prompt 追踪与对比能力。
记录失败案例	建一个"bad_cases.md"，每次发现 AI 翻车就记录，定期回头测新 prompt 能否解决。

阶段 2 结束自检

能用 5 段结构写出一个 200+ 字的 system prompt
至少使用过 5 种 prompt 技法 (few-shot / CoT / JSON / self-reflection / negative)
会用 function calling 让 AI 调用一个真实 API (如天气)
有一个 10+ 用例的 prompt 测试集，能跑回归
能解释什么是幻觉，并用 3 种方法降低它

目标：让 AI “读”你的私有数据。所有“AI 客服”、“AI 知识库”、“AI 笔记问答”本质都是 RAG。

RAG 完整 6 步链路

步骤	做什么	关键决策
1. Load	加载源文档：PDF / Word / Markdown / 网页 / 数据库	解析器选型（PyPDF/Unstructured/Llama Parse）
2. Split	把长文档切成小块（chunk）	切块大小与重叠（影响极大）
3. Embed	每个 chunk 用 embedding 模型转成向量	选哪个 embedding 模型
4. Store	向量存入向量数据库	ChromaDB / Qdrant / pgvector
5. Retrieve	用户提问 → 也转向量 → 找最相似的 top-k 个 chunk	top-k 数量、相似度阈值、重排
6. Generate	把检索到的 chunk 拼进 prompt 让 LLM 回答	prompt 结构、上下文组织

Embedding 模型对比（2026）

模型	维度	特点	推荐
text-embedding-3-small (OpenAI)	1536	英文最稳、便宜	英文/通用
text-embedding-3-large (OpenAI)	3072	更高质量、更贵	高精度场景
bge-large-zh-v1.5 (BAAI)	1024	中文最强、开源、可本地	中文首选
DeepSeek embedding	-	便宜、中文够用	国内项目
voyage-3 (Anthropic 推荐)	1024	准确率高、Claude 生态首选	Claude 用户

文档切块策略 4 种

固定长度	每 500 字一切，重叠 50 字。最简单，但可能切断句子语义。新手起步用。
按结构切	按标题/段落/章节切。Markdown/HTML 友好。语义边界清楚。
按语义切	用 embedding 计算相邻句子相似度，相似度突变处切。效果最好，成本最高。
递归切	LangChain 的 RecursiveCharacterTextSplitter：先按段落切，超长就按句子切。性价比之王。

切块黄金参数：chunk_size=500-1000 字，chunk_overlap=10-20%。小块召回精准但可能丢上下文；大块上下文全但检索准确率下降。需按业务测。

向量数据库选型

方案	部署	规模上限	推荐场景
ChromaDB	本地单机	< 100 万 chunk	入门/原型/个人项目（首选）
Qdrant	Docker / 云	亿级	生产环境、高性能、Rust 写的
pgvector	PostgreSQL 扩展	千万级	已有 PG 的项目，省一套服务
Milvus	K8s 集群	十亿级	大厂级别、运维成本高
FAISS	本地内存	千万级（内存够）	纯检索、无持久化、最快
Pinecone	SaaS	不限	懒得自部署、愿意付费、海外

RAG 调优 5 大杠杆

1. Chunk 策略	改切块大小与重叠，效果差异最大。优先调这个。
2. Top-K	检索几个 chunk 拼进 prompt？常用 3-5。多了噪声、少了信息不足。
3. 混合检索	向量检索 + 关键词检索（BM25）结合。中文场景普遍提升 10-20%。
4. Rerank	初步检索 top-20 → 用 reranker 重排取 top-3。bge-reranker-large 效果好。
5. Query 改写	用户问"价格多少" → AI 改写为"产品 X 的定价方案"。检索召回提升明显。

RAG 评测（RAGAS 框架）

Context Precision	检索到的 chunk 里有多少是真的相关的？衡量"检索质量"。
Context Recall	应该被检索到的关键 chunk，召回了几个？衡量"是否漏召"。
Faithfulness	生成答案是否忠于检索到的内容？衡量"幻觉"。
Answer Relevancy	答案是否切题？衡量"生成质量"。
工具	ragas (pip install ragas)，配合 GPT-4 / Claude 做评测员。

RAG 翻车排查表

症状	可能原因	解决方向
答非所问	检索召回错了	看 retrieved chunks，改切块/换 embedding
一本正经胡说	检索没找到，模型靠记忆编	Prompt 加"信息不足必须说不知道"
答案截断	max_tokens 太小 / context 超限	加大 max_tokens / 减少 top-k
英文 query 失败	中文 embedding 模型只懂中文	换多语言 embedding（如 bge-m3）

速度慢	每次都重新 embed 全部文档	向量结果持久化、批量预计算
同义词查不到	向量模型对术语不敏感	加同义词扩展、混合 BM25 检索

目标：让 AI 不只回答问题，还能“做事”——自主调工具、循环决策、完成多步任务。这是 2026-2028 最稀缺的能力。

Agent vs Chatbot 本质区别

Chatbot	一问一答，模型只输出文本。逻辑由人控制。RAG 也属于这一类。
Agent	模型自主决定下一步做什么，循环执行直到目标达成。包含“思考-行动-观察”循环。
举例	Chatbot：“帮我写邮件” → 输出文本。Agent：“帮我订一张明天去北京的机票” → 自动查航班/比价/预订/确认。
难度	Agent 难度比 Chatbot 高一个数量级，主要是“鲁棒性”——LLM 偶尔出错，循环会放大错误。

ReAct 模式 (Reasoning + Acting)

Agent 的基础范式。模型每一步输出：

Thought (思考)：我现在应该做什么？

Action (行动)：调用哪个工具，参数是什么？

Observation (观察)：工具返回了什么？

然后回到 Thought，循环直到任务完成。这就是 LangGraph / AutoGen / Claude Agent SDK 的底层共同模型。

Agent 框架对比

框架	定位	优势	劣势
LangChain	老牌、链式	生态全、教程多	抽象层重、版本不稳
LangGraph	LangChain 升级版	图状态机、可视化、调试好	学习曲线陡
CrewAI	多 Agent 协作	Role-Based 模式直观	生产化方案少
AutoGen	微软多 Agent	对话式编程范式	迭代快、文档跟不上
Claude Agent SDK	Anthropic 官方	与 Claude 深度集成、稳定	只支持 Claude
裸 Python	自己实现 ReAct	完全可控、零依赖	需手写工具管理

新手路径：裸 Python 写一个 ReAct 循环 (100 行) → 上 LangGraph (理解图状态) → 按需碰其他框架。

MCP 协议 (Model Context Protocol)

是什么	Anthropic 2025 年发布的 AI 工具调用标准。可以理解为“AI 界的 USB-C”。
为什么重要	一次开发的 MCP Server 能被所有支持 MCP 的客户端 (Claude Desktop / Cursor / IDE) 调用，复用性极强。
学多深	能写一个简单的 MCP Server (连接你的数据库/笔记/日历给 AI 用) 即可。半天足够。
资源	modelcontextprotocol.io 官方文档，Python/TypeScript SDK 都有。

六、模型本地化与微调（按需深化）

什么时候才需要这一章？1) 数据隐私敏感不能上云 2) 调 API 成本超过本地硬件成本 3) 想做特定领域的精细化模型。否则保持调 API 即可，性价比最高。

本地跑模型（Ollama 路线）

Ollama	ollama.com 一键安装。命令行 ollama run llama3 即可跑。零配置。
硬件门槛	7B 模型 → 8GB 内存可跑（量化版）；13B → 16GB；70B → 64GB。Mac M 系列特别友好。
推荐模型	Qwen2.5（中文最强开源）、Llama 3.3（综合）、DeepSeek-Coder（写代码）、Phi-3（小且快）。
用途	本地原型/隐私场景/无网环境。质量比 GPT-4 仍差，但够大部分场景用。
集成	Ollama 提供 OpenAI 兼容 API，把 base_url 改成 localhost:11434 即可无缝切换。

量化技术速览

为什么量化	把模型参数从 FP16 压到 INT8/INT4，显存占用 -50%~75%，质量损失 < 5%。让消费级显卡跑大模型。
GGUF	llama.cpp 生态格式，CPU/Mac 跑模型首选。Ollama 用的就是 GGUF。
AWQ / GPTQ	GPU 上的两种主流量化方案。AWQ 质量略好，GPTQ 速度略快。vLLM 都支持。
选择建议	本地玩 GGUF；GPU 部署用 AWQ。新手用 Ollama 就够，不需要管这些细节。

微调（Fine-tuning）何时该做

不该微调	Prompt 工程能解决的别微调。90% 的问题加 few-shot 就够，微调成本是 prompt 的 100 倍。
该微调的场景	1) 固定格式输出且 prompt 怎么调都不稳 2) 特定领域术语/风格 3) 模型尺寸要做小
LoRA / QLoRA	只训练 1-5% 的参数，单卡可微调 7B 模型。Hugging Face PEFT 库 + Unsloth 加速框架。
数据量	至少 500-2000 条高质量训练样本。数据质量比数量重要 10 倍。
推荐学习	Hugging Face 官方课程 NLP Course + Unsloth GitHub 示例（最快上手路径）。

七、AI 工程化生产实践

demo 到产品的 90% 工作量都在这里。也是区分"AI 玩家"和"AI 工程师"的分水岭。

API Key 安全管理

永远不要放前端	API Key 一旦泄漏，攻击者可烧光你的账户。所有调用必须从后端转发。
环境变量	本地用 .env + python-dotenv。 .env 加入 .gitignore。生产用云厂商的 secrets manager。
密钥轮换	每个项目独立的 Key，每 30-90 天轮换。一旦泄漏立即吊销，不要犹豫。
调用频率限制	后端用 Redis 做 rate limit（IP/用户级别），防止有人刷你的 API 余额。

稳定性：重试、降级、超时

指数退避重试	429（限流）和 5xx → 指数退避（1s/2s/4s/8s），最多 3-5 次。tenacity 库现成的。
超时设置	API 调用必须设 timeout（推荐 60-120s），永远不要无限等。流式输出要做心跳检测。
模型降级	主模型挂了自动切备用模型（Claude → DeepSeek → 本地 Ollama）。多模型代理网关推荐 LiteLLM。
熔断	某个模型连续失败 10 次 → 拉黑 5 分钟，避免持续失败拖垮整个服务。

成本监控

Token 统计	每次调用记录 prompt_tokens + completion_tokens。日/周聚合到数据库。
成本告警	日支出超过预算阈值 → 钉钉/邮件告警。否则一个死循环 bug 能烧掉一个月预算。
缓存	相同 prompt 直接返回缓存结果，可省 30-70%。Anthropic prompt caching 原生支持。
小模型分流	简单任务用小模型（DeepSeek/Haiku），复杂任务才用大模型。成本下降 5-10 倍。

可观测性（Observability）

LangSmith	LangChain 官方，记录每次 LLM 调用的完整链路。免费额度够个人项目用。
LangFuse	开源版 LangSmith，可自部署。Docker 一键起。隐私敏感项目首选。
日志结构化	至少记录：input、output、tokens、latency、model、user_id、trace_id。便于回溯。
错误聚类	Sentry 收集异常，按错误类型聚合。AI 产品的 bug 模式与传统软件不同，要单独看。

八、踩坑经验集（12 个真实问题）

1. Token 超限 (Context too long)	模型上下文窗口有限（8K/32K/128K）。长对话/长文档很容易爆。解：定期总结历史对话、用 RAG 替代全量塞、改用长上下文模型（Claude 200K / Gemini 1M）。
2. 中文乱码	编码问题。读 PDF/网页时尤其常见。解：明确指定 encoding=utf-8，用 chardet 自动检测。PDF 读取优先用 PyMuPDF / pdfplumber，不要用 PyPDF2。
3. JSON 解析失败	模型输出的 JSON 偶尔不合法（多个逗号、漏引号、混 markdown 代码块）。解：1) 用 response_format=json_object 2) json_repair 库容错解析 3) 失败重试最多 3 次。
4. 流式输出中断	网络抖动 / 服务端超时 → stream 卡住。解：客户端做心跳检测，3 秒无数据自动重连。前端 SSE 加 retry 参数。
5. 上下文遗忘	多轮对话越长，模型越倾向忽略早期内容（lost-in-the-middle 现象）。解：关键约束放最后、定期总结历史、把核心信息放 system prompt 而非历史消息。
6. 速度太慢	一个复杂 prompt 跑 30 秒。解：1) 流式输出立刻有反馈 2) 拆并行子任务 3) 缓存 4) 换更快的模型（Haiku/DeepSeek）。
7. RAG 答非所问	检索召回错了或没召回到。解：日志打印 retrieved_chunks，肉眼看对不对。再调切块/embedding/top-k。
8. Prompt 改了一处全崩	没有版本管理 + 没有测试集 → 每次改 prompt 都是赌博。解：建 10+ 用例的测试集，每次改完跑一遍。
9. Function Calling 参数错	模型生成的工具参数不符合 schema。解：1) description 写得像说明书 2) 用 Pydantic 强校验 + 错误重试 3) 给 few-shot 示例。
10. 账单爆炸	一个 bug 写循环调 API，一晚上烧光 500 美元（真实案例）。解：1) 必设日额度上限 2) 加 rate limit 3) 早期用 DeepSeek 等便宜模型试错。
11. Embedding 维度不匹配	换了 embedding 模型却用旧向量库 → 检索全错。解：换 embedding 必须重新 embed 全部文档。建库时记录用的什么模型。
12. 模型升级反而变差	GPT-4 → GPT-4o 后某些 case 退化。解：升级前用测试集回归，新模型不一定全方位更好。固化 model 版本号，不要用别名。

九、6 个进阶练手项目（按难度递增）

每个项目都覆盖一项核心技能。建议至少完成 4 个，把 GitHub 仓库做成你的简历。

1. 智能命令行助手	难度：★ 周期：3 天 技能：API 基础 + 流式输出 CLI 工具，支持多轮对话、保存历史、切换模型、导出对话为 Markdown。
2. 个人 PDF 问答机器人	难度：★★ 周期：1 周 技能：RAG 入门 上传 PDF → 切块 → embedding → ChromaDB → 提问。前端 Streamlit / Gradio 一页搞定。
3. 邮件分类与自动回复	难度：★★ 周期：1 周 技能：JSON 输出 + Function Calling 接 IMAP 拉邮件 → AI 分类（重要/广告/账单）→ 重要邮件生成草稿。展示 AI 集成现有系统的能力。
4. 微信公众号文章助手	难度：★★★ 周期：2 周 技能：Prompt Chaining + 工程化 输入选题 → 大纲生成 → 段落填充 → 配图建议 → 排版。完整产品形态。
5. 代码库 AI 问答	难度：★★★★ 周期：3 周 技能：进阶 RAG + 多模态 指向 GitHub 仓库 → 自动 clone → 解析代码结构 → 函数级 embedding → 自然语言问“这个函数干嘛的”。
6. 自任务 Agent	难度：★★★★★ 周期：4 周 技能：Agent + MCP + 多工具 “帮我研究 X 主题”→ AI 自主搜索/读论文/做笔记/总结成报告。集成 Web 搜索、爬虫、笔记存储等多工具。

关键原则：项目不是越多越好。把 1-2 个做到能上线、有真实用户用、有 README 和演示视频，比做 10 个半成品强 100 倍。

十、精选资源清单（少而精）

原则：宁可看一份资源 3 遍，不要囤 30 份资源看 1 遍。下列每一项都经过实战验证，按推荐顺序排列。

必看视频（按顺序）

1. Andrej Karpathy 「Intro to LLMs」	YouTube，1 小时。建立对 LLM 最重要的直觉，免费。看一遍胜过 10 本书。
2. Andrej Karpathy 「Let's build GPT」	YouTube，4 小时。从 0 用 PyTorch 实现一个 mini-GPT。想理解原理这一个就够了。
3. fast.ai 《Practical Deep Learning》Part 1	course.fast.ai。"自上而下"教学法，先做出东西再补理论，与本指南理念完全一致。

必读文档

Anthropic Prompt Engineering Guide	docs.anthropic.com 的 prompt-engineering 章节。工业界最系统的 prompt 教程。中文翻译版也很多。
OpenAI Cookbook	github.com/openai/openai-cookbook。各种实战示例，所有典型用法都能找到代码。
LangChain / LangGraph 文档	python.langchain.com。不必通读，需要某个功能再查。Conceptual Guide 那一章值得读。
DeepSeek API 文档	api-docs.deepseek.com。中文最友好，新手第一站。

GitHub 必关注

mlabonne/llm-course	github 上 40k star 的 LLM 综合学习仓库。结构化路线图 + Colab 实战。
langchain-ai/langchain	不一定都用，但 examples 目录是 prompt/agent 的活教材。
Unsloth-ai/unsloth	微调最快的开源框架。要做 LoRA 微调时直接抄它的 notebook。
Anthropic / claude-cookbooks	Claude 官方 cookbook。Tool Use / RAG / Agent 都有规范示例。
ggerganov/llama.cpp	本地跑模型的底层引擎。即使用 Ollama，也建议了解 llama.cpp 的存在。

必读论文（只 3 篇）

Attention Is All You Need (2017)	Transformer 开山论文。即使不读全文，也要看图 1 那张架构图。
Retrieval-Augmented Generation (2020)	RAG 概念正式提出。理解为什么要做 RAG。

ReAct: Synergizing Reasoning and Acting (2022)

所有 Agent 框架的理论基础。读完你会对 LangGraph 的设计豁然开朗。

中文社区与博客

宝玉的微博	中文 AI 信息最快源头之一。每天 5 分钟扫一遍即可。
十字路口 Crossing	中文 AI 播客，每周更新，质量稳定。通勤听。
量子位 / 机器之心	中文 AI 资讯，看标题即可，深度内容不多。
魔搭社区 (ModelScope)	阿里开源模型平台。国内场景的中文模型基本都在这。

最后一句话：AI 学习曲线最大的敌人是"看似学了很多，实际什么都没做出来"。从今天开始，每一周必须有一个可以发到朋友圈的产出。哪怕只是一个 30 行的脚本——只要它在解决你自己的真实问题。

本指南由 Claude (Anthropic) 为你定制生成 · 2026 年 5 月 · 配套《个人定制学习路线》